

RAG classique vs neoteem-brain

Document de reference interne Neoteem — Avril 2026

En une phrase

Le RAG injecte des morceaux de texte dans le prompt de l'IA. neoteem-brain donne a Claude Code des outils pour naviguer la connaissance lui-meme.

Comment Claude Code recoit le contexte

C'est LA difference fondamentale. Les deux approches donnent du contexte a Claude Code, mais pas de la meme maniere.

Avec RAG : contexte passif (on lui donne des morceaux)

```
None
Developpeur pose une question a Claude Code
|
Le systeme RAG cherche dans la vector DB
|
5 chunks de ~500 tokens sont recuperes automatiquement
|
Ces chunks sont INJECTES dans le prompt de Claude Code
(il ne choisit pas lesquels, il les recoit passivement)
|
Claude Code lit les chunks et essaie de repondre
```

Claude Code ne controle rien. Il recoit 5 morceaux de texte decoupees mecaniquement et doit se debrouiller avec. S'il lui manque du contexte, il ne peut pas aller le chercher. S'il recoit un chunk inutile, il a quand meme consomme les tokens.

C'est comme donner a quelqu'un 5 pages arrachees au hasard d'un livre et lui demander de repondre a une question.

Avec neoteem-brain : contexte actif (il va chercher lui-meme)

None

Developpeur pose une question a Claude Code

|
Claude Code DECIDE de chercher dans le vault

|
search:context query="..." --> snippets cibles (~300 tokens)
Claude Code LIT les snippets et CHOISIT quelles notes lire

|
read file="..." --> seulement les 1-3 notes pertinentes

|
Claude Code VOIT les [[wikilinks]] dans la note
Il DECIDE de suivre un lien ou non selon le besoin

|
backlinks file="..." --> il decouvre les notes liees

|
Claude Code construit sa comprehension pas a pas
comme un humain qui navigue dans une documentation

Claude Code est un agent actif. Il cherche, il lit, il navigue, il decide quand il en a assez. S'il lui manque du contexte, il suit un wikilink. S'il a assez d'info, il s'arrete. Il ne consomme que les tokens necessaires.

C'est comme donner a quelqu'un l'acces a une bibliotheque bien rangee avec un index et le laisser trouver ce dont il a besoin.

Impact concret sur la qualite des reponses

Situation	RAG (contexte passif)	neoteem-brain (contexte actif)
Question simple ("c'est quoi un ADF ?")	Recoit 5 chunks dont 3 hors sujet. Repond avec du bruit. ~2500 tokens	search:context trouve la bonne note en 1 appel. ~300 tokens
Question complexe ("comment fonctionne la compensation syndic-gerance ?")	Recoit des chunks partiels de 3 notes differentes. Ne peut pas reconstituer la chaine.	Suit les wikilinks : regle metier → fonction PG → table → ecran Lojii. Chaine complete

Situation	RAG (contexte passif)	neoteem-brain (contexte actif)
Question cross-domaine ("impact d'une resiliation de bail sur la compta")	Les chunks viennent probablement d'un seul domaine. L'autre domaine est absent	backlinks file="bail" → trouve les notes compta liees. Traverse les domaines
Info obsolete ("la table t_xyz...")	Le chunk obsolete remonte. Pas de signal que c'est perime	La note a un callout [!warning] Obsolete + lien vers le remplacement
Suivi de conversation ("et cote frontend ?")	Nouveau retrieval, re-embedding, re-injection de chunks. Perd le contexte precedent	Claude Code a deja lu la note. Il suit le wikilink vers la note frontend. Gratuit

Architecture comparee

RAG classique

```
None
Documents bruts (PDF, pages web, code, Confluence...)
|
Chunking (decoupage en morceaux de ~500 tokens)
|
Embedding API (OpenAI, Cohere...) --> vecteurs numeriques
|
Vector DB (Pinecone, Weaviate, ChromaDB, pgvector...)
|
Query --> embedding --> recherche cosine --> top-K chunks
|
Chunks injectes dans le prompt LLM --> reponse
```

neoteem-brain

```
None
Notes Obsidian curatees (682 fichiers structures)
|
Index natif Obsidian (full-text, ~70 000x plus efficace
```

```
que lire le filesystem directement)
|
CLI search:context --> snippets cibles (0 token de recherche)
|
Read cible sur 1-3 notes pertinentes seulement
|
Wikilinks --> navigation vers les notes liees si besoin
|
Reponse avec contexte complet (chaine front --> API --> PG)
```

Cout en tokens par requete

Etape	RAG classique	neoteem-brain
Recherche	Embedding de la query (~50 tokens) + appel API embedding payant	search:context via CLI = 0 token LLM
Resultats injectes	5 chunks x ~500 tokens = ~2 500 tokens dans le prompt	Snippets cibles = ~200-500 tokens
Lecture approfondie	Pas possible, on n'a que les chunks	read sur 1-3 notes = ~500-3 000 tokens (si necessaire)
Contexte relationnel	Aucun, les chunks sont isolees	backlinks + wikilinks = relations gratuites
Total par requete	~3 000 - 12 000 tokens + cout embedding	~700 - 4 000 tokens, 0 cout externe

Sur 100 queries par jour

Metrique	RAG classique	neoteem-brain
Tokens LLM	~300K - 1.2M	~70K - 400K
Cout embedding externe	~\$0.01-0.05/jour	\$0
Cout vector DB	\$20-100/mois (heberge)	\$0
Economie tokens	baseline	~60-70% de reduction

Qualite des reponses

Contexte

RAG : Chunks de 500 tokens decoupees mecaniquement. Si la reponse chevauche 2 chunks, elle est perdue.

neoteem-brain : Notes completes et structurees. Frontmatter avec metadonnees. Wikilinks vers les notes liees.

Relations entre concepts

RAG : Aucune relation. Chaque chunk est isole. "La table t_bail" dans un chunk n'est pas reliee au chunk sur "le processus de resiliation".

neoteem-brain : Wikilinks bidirectionnels. La commande `backlinks file="t-bail"` trouve toutes les regles metier, fonctions PG et FAQ qui parlent de cette table. Les relations sont explicites et navigables.

Chaine complete front-to-back

RAG : Impossible. Le RAG retourne le chunk le plus proche vectoriellement, pas la chaine d'appels.

neoteem-brain : La chaine complete est documentee et liee. On peut suivre : bouton Lojii --> route Vue.js --> endpoint Go --> fonction PG. Chaque maillon est une note avec des wikilinks vers les maillons adjacents.

Fraicheur de l'information

RAG : Necessite un re-embedding a chaque modification de document. Si oublie, le RAG retourne une reponse obsolete silencieusement.

neoteem-brain : Notes mises a jour directement. Le champ `derniere-maj` dans le frontmatter trace la derniere modification. Un callout `[!warning] Obsolete` est ajoute si un concept est supprime, avec un lien vers le remplacement.

Contradictions

RAG : Peut retourner 2 chunks contradictoires sans le detecter.

neoteem-brain : L'agent sync-checker audite les contradictions. Les wikilinks permettent de croiser les sources et identifier les incohérences.

Hallucinations

RAG : Le LLM peut "compléter" un chunk partiel avec des informations inventées.

neoteem-brain : Les notes sont curatées et vérifiées par les équipes. Chaque note a un champ **sources** dans le frontmatter qui trace l'origine (Confluence, code, Jira).

Exemple concret

Question : "C'est quoi le calcul des charges en copropriété ?"

Avec RAG

1. Embedding de la question --> vecteur 1536 dimensions
2. Recherche cosinus dans 5000 chunks --> top 5 résultats :
 - Chunk 42 : "...les charges sont réparties selon les tantièmes..." (tronqué au milieu)
 - Chunk 187 : "...fonction f_calc_charges prend en paramètre..." (technique brut)
 - Chunk 203 : "...le copropriétaire reçoit un appel de fonds..." (contexte partiel)
 - Chunk 89 : "...CREATE TABLE t_charge_copro..." (DDL brut inutile)
 - Chunk 301 : "...procédure de vote du budget en AG..." (hors sujet)
3. Le LLM reçoit ces 5 chunks décousus et essaie de construire une réponse cohérente

Résultat : réponse partielle, mélange technique et métier, contexte perdu. ~5 000 tokens consommés.

Avec neoteem-brain

1. `search:context query="calcul charges copropriete" limit=5` --> 3 notes pertinentes identifiées en ~300 tokens
2. `read file="adf-appel-de-fonds"` (note métier structurée) --> Processus complet : vote budget --> répartition tantièmes --> ADF trimestriel, ~800 tokens
3. Wikilinks dans la note : `[[f-calc-charges-copro]]`, `[[t-charge-copro]]` --> On sait que la fonction PG et la table existent. Pour une question métier, pas besoin de les lire.

4. `backlinks file="adf-appel-de-fonds" --> Trouve`
`[[proc-campagne-adf-syndic]]` (procedure support avec navigation ecran), ~200 tokens

Resultat : reponse complete avec la chaine metier, la navigation dans Lojii et la procedure. ~1 300 tokens consommés.

Infrastructure

Element	RAG classique	neoteem-brain
Base de donnees	Vector DB (Pinecone, Weaviate, ChromaDB, pgvector) a installer et maintenir	Aucune. Fichiers markdown + git
Pipeline d'embedding	Script d'ingestion + chunking + embedding + upsert a chaque modification	Aucun. Les notes sont ecrites directement dans le vault
API externe	OpenAI Embeddings ou Cohere (payant, dependance cloud, latence)	Aucune. Tout est local
Maintenance	Re-embedding quand les docs changent, monitoring du vector DB, gestion des versions de chunks	git pull + Obsidian ouvert
Debugging	Boite noire. "Pourquoi cette reponse ?" --> inspecter les chunks retournes, les scores cosine, le prompt assemble	Transparent. Ouvrir Obsidian, lire la note, voir les wikilinks
Backup	Export vector DB + documents sources separement	git push (deja fait, versionne)
Cout mensuel	\$20-100+ (vector DB heberge) + \$5-20 (API embeddings)	\$0

Element	RAG classique	neoteem-brain
Securite	Donnees envoyees a une API externe pour l'embedding	100% local, chiffable, aucune donnee externalisee

Couverture semantique : aliases vs embeddings

Le seul avantage du RAG : la recherche semantique. Si quelqu'un cherche "impayees" mais que la note s'appelle "relances-locataires", les embeddings trouvent le lien semantique.

Notre solution : les aliases dans le frontmatter de chaque note.

```
None
# relances-locataires.md
aliases:
  - "impayees"
  - "loyers impayees"
  - "recouvrement locataire"
  - "RAL"
  - "relance impaye"
  - "locataire qui paye pas"
```

L'index Obsidian cherche dans les aliases. Le resultat est deterministe (100% fiable) contre les embeddings qui sont probabilistes (le score cosine peut rater, surtout en francais ou les modeles d'embedding sont moins performants).

Sources de vocabulaire :

- [glossaire.md](#) : 70+ termes metier officiels Neoteem
- [glossaire-support.md](#) : reformulations simples pour les equipes support

Chaque note a au minimum 3 aliases, sans maximum, couvrant : synonyme metier, abreviation, vocabulaire support, terme technique.

Quand le RAG serait quand meme meilleur

Situation	Pourquoi RAG gagnerait	Notre cas ?
10 000+ documents	L'index keyword ne suffit plus, les aliases deviennent ingerables a maintenir	Non. 682 notes, seuil estime a 2000-3000
Documents non structures (PDF bruts, emails, logs)	Le chunking est le seul moyen de les rendre cherchables	Non. Notes curatees en markdown structure
Recherche multi-langue	Les embeddings multilingues trouvent cross-langue	Non. Tout est en francais
Utilisateurs anonymes sans vocabulaire metier	Impossible de deviner tous les aliases necessaires	Non. Notre equipe connait le vocabulaire
Gigaoctets de donnees	L'index keyword sature, les embeddings scalent mieux	Non. ~682 fichiers markdown legers

Les 4 agents du vault

Le vault est maintenu par 4 agents specialises :

Agent	Role	Quand
repo-analyzer	Lit les repos de code (Go, TS, Vue, Python, PG), cree des notes techniques dans le vault	Quand un nouveau repo ou module doit etre documente
vault-enricher	Lit Confluence (espaces NeolA et DTI), cree des notes dans le vault	Quand de la documentation Confluence doit etre importee
vault-linker	Ajoute les wikilinks bidirectionnels, met a jour les MOCs (Maps of Content)	Automatiquement apres chaque creation de notes

Agent	Role	Quand
sync-checker	Audite l'integrite : frontmatter complet, liens valides, pas d'orphelins	Automatiquement apres les batch de creation

Un pipeline conditionnel orchestre ces agents : les gros batch declenchent tous les agents, les petites modifications ne declenchent que le minimum necessaire.

Optimisations token-smart (avril 2026)

Optimisation	Economie	Description
search:context d'abord	~60-70% sur les queries	Retourne des snippets, pas le fichier entier. Lecture complete seulement si necessaire
Pipeline conditionnel	~40% sur les ecritures	Un append ne declenche plus 2 agents. Seuls les batch declenchent le pipeline complet
Check avant auto-creation	~30% sur le support	Verifie si une FAQ existe avant d'en creer une nouvelle. Pas de doublons
Correction sur place	Prevention	Info obsolete = mise a jour de la note existante + propagation aux backlinks. Pas de nouvelle note
Aliases obligatoires	Prevention	Au minimum 3 aliases par note couvrant synonymes, abreviations, vocabulaire support et technique

Resume

Dimension	RAG classique	neoteem-brain
Comment l'IA recoit le contexte	Passif : chunks injectes dans le prompt	Actif : l'IA cherche, lit, navigue elle-meme
Tokens par query	3 000 - 12 000	700 - 4 000
Infrastructure	Vector DB + embedding API + pipeline	Git + Obsidian
Cout mensuel	\$25 - 120+	\$0
Qualite contexte	Chunks isolees	Notes liees par wikilinks
Debugging	Boite noire	Transparent (ouvrir Obsidian)
Fraicheur	Re-embedding necessaire	Mise a jour directe
Securite	Donnees externalisees	100% local
Recherche semantique	Native (embeddings)	Aliases deterministes
Scalabilite	Illimitee	Bonne jusqu'a ~2000-3000 notes